

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 2
MODUL 18
“MESIN ABSTRAK”



DISUSUN OLEH:
Andromeda Alika Ramadhani
109082500156
S1 IF-13-02

DOSEN:
Wahyu Andi Saputra, S.Pd., M.Eng

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2024/2025

DASAR TEORI

Mesin abstrak merupakan sebuah model teoretis yang digunakan dalam dunia ilmu komputer untuk mensimulasikan cara kerja pemrosesan data sekuensial (berurutan) tanpa bergantung pada perangkat keras fisik tertentu.

Komponen utama mesin karakter:

1. Pita Karakter (Tape) adalah tempat penyimpanan teks yang akan diproses. Isi teks ini dibaca satu per satu dari kiri ke kanan. Setiap teks wajib diakhiri dengan tanda titik (.). Kalau di dalam pita hanya ada tanda titik saja, maka dianggap sebagai pita kosong.
2. Jendela (Window) adalah bagian pembaca yang hanya bisa melihat satu karakter dalam satu waktu. Karakter yang sedang terbaca oleh jendela ini disebut Current Character (CC).
3. EOP (End of Tape) adalah penanda berupa nilai boolean (true/false). EOP ini akan menyala (true) ketika pembaca menemukan tanda titik (.), yang artinya proses pembacaan teks sudah selesai.

Operasi primitif mesin karakter:

1. START: Prosedur ini digunakan untuk menjalankan atau menyalakan mesin. Caranya dengan memasukkan teks ke dalam pita, lalu menempatkan pembaca di posisi awal (indeks pertama). Setelah itu, karakter pertama langsung dibaca sebagai CC (Current Character). Jika karakter pertama bukan titik (.), maka status EOP akan bernilai false.
2. ADV (Advance): Prosedur ini berfungsi untuk menggeser posisi pembacaan satu langkah ke kanan. Setiap kali dijalankan, CC akan diperbarui menjadi karakter berikutnya. Jika karakter yang terbaca adalah titik (.), maka proses pembacaan berhenti dan EOP otomatis menjadi true.
3. CC: Fungsi yang digunakan untuk mengambil atau menampilkan karakter yang sedang dibaca oleh jendela pada posisi saat ini.
4. EOP: Fungsi yang digunakan untuk mengecek apakah pembacaan sudah sampai akhir pita. Jika bernilai true, berarti mesin sudah mencapai akhir dan tidak bisa digerakkan lagi.

CONTOH SOAL

1. Latihan1

Source Code:

Output:

Deskripsi Program:

SOAL LATIHAN

Statement Mesin Domino dan Mesin Abstrak Karakter

1. Soal1

Source Code:

```
package main

import (
    "fmt"
    "math/rand"
    "time"
)

// 1.a) Tipe Data Kartu Domino
type Domino struct {
    KiriKartu  int //sisi kiri kartu
    KananKartu int //sisi kanan kartu
    TotalKartu int //nilai total kartu
    IsBalak    bool //apakah kartu ini balak (kembar)
    atau bukan
}

type Dominoes struct {
    kartu    [28]Domino
    sisaKartu int
}

func inisiasiDomino() Dominoes {
    var d Dominoes
    var idx int = 0
    var i, j int

    for i = 0; i <= 6; i++ {
```

```

        for j = i; j <= 6; j++ {
            d.kartu[idx] = Domino{
                KiriKartu:  i,
                KananKartu: j,
                TotalKartu: i + j,
                IsBalak:    i == j,
            }
            idx++
        }
    }

    d.sisaKartu = 28
    return d
}

// 1.b) Prosedur kocok kartu
func kocokKartu(d *Dominoes) {
    var i, j int

    for i = d.sisaKartu - 1; i > 0; i-- {
        j = rand.Intn(i + 1)
        d.kartu[i], d.kartu[j] = d.kartu[j], d.kartu[i]
    }
    //menukar posisi kartu
}

// 1.c) Fungsi ambil kartu
func ambilKartu(d *Dominoes) Domino {
    if d.sisaKartu <= 0 {
        fmt.Println("Tumpukan kartu habis")
        return Domino{}
    }
}

```

```
        var kartuDiambil Domino
        kartuDiambil = d.kartu[d.sisaKartu-1]
        d.sisaKartu--

        return kartuDiambil
    }

    // 1.d) Fungsi gambar kartu
    func gambarKartu(d Domino, suit int) int {
        if suit == 1 {
            return d.KiriKartu
        } else if suit == 2 {
            return d.KananKartu
        }
        return -1
    }

    // 1.e) Fungsi nilai kartu
    func nilaiKartu(d Domino) int {
        return d.TotalKartu
    }
```

The screenshot shows a Go IDE with two tabs: 'domino.go' and 'mesinAbstrakKarakter.go'. The 'domino.go' tab is active, displaying Go code for a domino game. The code defines a 'Domino' struct with attributes 'KiriKartu', 'KananKartu', 'TotalKartu', and 'IsBalak'. It also defines a 'Dominoes' struct with an array 'kartu' and an integer 'sisaKartu'. A function 'inisiasiDomino()' is shown, which initializes the 'kartu' array with 28 unique domino combinations. A terminal window on the right shows the output of the program: '109082500156', 'IF-13-02', and 'Andromeda Alike Ramadhani'.

```
1 package main
2
3 import (
4     "fmt"
5     "math/rand"
6     "time"
7 )
8
9 // 1.a) Tipe Data Kartu Domino
10 type Domino struct {
11     KiriKartu int //sisi kiri kartu
12     KananKartu int //sisi kanan kartu
13     TotalKartu int //nilai total kartu
14     IsBalak bool //apakah kartu ini balak (kembar) atau bukan
15 }
16
17 type Dominoes struct {
18     kartu [28]Domino
19     sisaKartu int
20 }
21
22 func inisiasiDomino() Dominoes {
23     var d Dominoes
24     var idx int = 0
25     var i, j int
26
27     for i = 0; i <= 6; i++ {
28         for j = i; j <= 6; j++ {
29             d.kartu[idx] = Domino{
```

Output:

```
109082500156
IF-13-02
Andromeda Alike Ramadhani
```

Output:

Deskripsi Program:

Pada program ini, bagian awal digunakan untuk mendefinisikan struktur data yang mewakili kartu domino. Tipe data Domino digunakan untuk menyimpan satu kartu domino yang terdiri dari empat atribut, yaitu KiriKartu dan KananKartu sebagai nilai pada kedua sisi kartu, TotalKartu sebagai hasil penjumlahan kedua sisi, serta IsBalak bertipe boolean untuk menandai apakah kartu tersebut merupakan kartu balak (kedua sisinya bernilai sama). Selain itu, terdapat tipe data Dominoes yang digunakan untuk menyimpan satu set kartu domino. Tipe ini memiliki array berkapasitas 28 elemen sesuai jumlah kartu domino standar dan variabel sisaKartu untuk mencatat jumlah kartu yang masih tersedia di dalam tumpukan.

Pembuatan seluruh kartu dilakukan melalui fungsi inisiasiDomino(). Fungsi ini menggunakan dua perulangan bersarang untuk menghasilkan semua kombinasi kartu domino dari nilai 0 sampai 6. Agar tidak ada kartu yang sama dalam urutan terbalik, seperti (1,2) dan (2,1), perulangan kedua selalu dimulai dari nilai perulangan pertama. Dengan cara ini diperoleh 28 kombinasi kartu unik yang kemudian disimpan ke dalam array. Setelah semua kartu dibuat, nilai sisaKartu diatur menjadi 28 sebagai tanda bahwa seluruh kartu masih tersedia.

Untuk mendukung jalannya permainan, dibuat beberapa subprogram dasar. Prosedur kocokKartu() digunakan untuk mengacak urutan kartu. Pengacakan dilakukan dengan menukar posisi kartu pada indeks tertentu dengan kartu pada indeks acak yang dihasilkan oleh fungsi rand.Intn().

Selanjutnya, fungsi ambilKartu() digunakan untuk mengambil satu kartu dari tumpukan. Mekanismenya mirip seperti struktur data stack, yaitu mengambil kartu pada

posisi paling atas (sisakartu - 1) lalu mengurangi nilai sisaKartu sebanyak satu untuk menandakan kartu tersebut sudah keluar dari deck.

Selain itu, terdapat dua fungsi pendukung, yaitu gambarKartu() dan nilaiKartu(). Fungsi gambarKartu() digunakan untuk mengambil nilai pada sisi kiri atau kanan kartu sesuai kebutuhan, sedangkan nilaiKartu() digunakan untuk mengembalikan total nilai dari sebuah kartu domino.

2. Soal2

Source Code:

```
// 2.a) Prosedur gali kartu

func galiKartu(d *Dominoes, target Domino, kartuHasil
*Domino, ditemukan *bool) {

    var kartuSekarang Domino

    *ditemukan = false

    for d.sisaKartu > 0 && !*ditemukan {

        kartuSekarang = ambilKartu(d)

        //cek suit yang cocok dengan target

        if kartuSekarang.KiriKartu == target.KiriKartu
|| kartuSekarang.KiriKartu == target.KananKartu ||

            kartuSekarang.KananKartu ==
target.KiriKartu || kartuSekarang.KananKartu ==
target.KananKartu {

                *kartuHasil = kartuSekarang

                *ditemukan = true

            }

        }

    }

}

// 2.b) Fungsi sepasang kartu

func sepasangKartu(d1, d2 Domino) bool {

    var total int

    total = nilaiKartu(d1) + nilaiKartu(d2)

    return total == 12

}
```


$\left. \begin{array}{l} \text{---} \\ \text{---} \\ \text{---} \end{array} \right\}$

```
80 // 2.a) Prosedur gali kartu
81 // 2.a) Prosedur gali kartu
82 func galiKartu(d *Dominoes, target Domino, kartuHasil *Domino, ditemukan *bool) {
83     var kartuSekarang Domino
84     *ditemukan = false
85
86     for d.sisaKartu > 0 && !*ditemukan {
87         kartuSekarang = ambilKartu(d)
88
89         //cek suit yang cocok dengan target
90         if kartuSekarang.KiriKartu == target.KiriKartu || kartuSekarang.KiriKartu == target.KananKartu ||
91            kartuSekarang.KananKartu == target.KiriKartu || kartuSekarang.KananKartu == target.KananKartu {
92             *kartuHasil = kartuSekarang
93             *ditemukan = true
94         }
95     }
96 }
97
98 // 2.b) Fungsi sepasang kartu
99 func sepasangKartu(d1, d2 Domino) bool {
100     var total int
101     total = nilaiKartu(d1) + nilaiKartu(d2)
102     return total == 12
103 }
104
105 // 3. Implementasi permainan Gapleh
```

Output:

Deskripsi Program:

Subprogram pertama adalah prosedur `galiKartu()` yang digunakan untuk mencari kartu tertentu di dalam tumpukan. Parameter tumpukan `kartu (d)`, `kartuHasil`, dan ditemukan dikirim menggunakan mekanisme *pass-by-reference* sehingga perubahan yang terjadi di dalam prosedur dapat langsung memengaruhi variabel pada program utama. Pada awal prosedur, variabel ditemukan diatur menjadi `false`. Selanjutnya, program melakukan pencarian menggunakan perulangan selama masih ada kartu di dalam tumpukan dan kartu yang dicari belum ditemukan. Pada setiap perulangan, satu kartu diambil dari tumpukan menggunakan fungsi `ambilKartu()`. Setelah itu, program membandingkan nilai pada kedua sisi kartu yang diambil dengan kartu target. Pengecekan dilakukan untuk semua kemungkinan pasangan sisi kartu. Jika ditemukan kecocokan, kartu tersebut disimpan ke `kartuHasil`, nilai ditemukan diubah menjadi `true`, dan proses pencarian langsung berhenti.

Subprogram kedua adalah fungsi `sepasangKartu()` yang digunakan untuk memeriksa apakah dua kartu domino membentuk pasangan yang valid. Fungsi ini menerima dua buah kartu domino sebagai parameter dan menghasilkan nilai bertipe boolean. Di dalam fungsi, dibuat variabel `total` untuk menyimpan hasil penjumlahan nilai kedua kartu. Perhitungan nilai kartu dilakukan menggunakan fungsi `nilaiKartu()` agar kode tetap rapi dan modular. Setelah `total` nilai diperoleh, program memeriksa apakah hasil penjumlahannya sama dengan 12. Jika sama, fungsi mengembalikan nilai `true`, sedangkan jika tidak sama maka mengembalikan nilai `false`.

3. Soal3

Source Code:

```

// 3. Implementasi permainan Gapleh

func main() {

    rand.Seed(time.Now().UnixNano())

    var skorTotal int = 0

    var totalRonde int = 0

    var i, skorRonde, keputusan, indeksKartu,
    ujungKanan, ujungKiri int

    var kartuDipakai [7]bool

    var rondeSelesai, keluarGame bool

    var tumpukanKartu Dominoes

    var kartuTanganPemain [7]Domino

    var kartuMulai, kartuPilihan Domino

    var rantaiKartu []Domino

    var tingkatKemenangan float64

    for {

        fmt.Printf("Your score is %d/%d \t\t At the
beginning scores=%d, round=%d\n", skorTotal,
totalRonde, skorTotal, totalRonde)

        fmt.Println("Dealing... \t\t\t Shuffle the
tiles for a new game")

        //1. Kocok kartu

        tumpukanKartu = inisiasiDomino()

        kocokKartu(&tumpukanKartu)

        //2. Bagi 7 kartu ke pemain

        for i = 0; i < 7; i++ {

            kartuTanganPemain[i] =
ambilKartu(&tumpukanKartu)

        }
    }
}

```

```

//3.Buka satu kartu dari tumpukkan untuk
memulai

kartuMulai = ambilKartu(&tumpukanKartu)
rantaiKartu = []Domino{kartuMulai}

for i = 0; i < 7; i++ {
    kartuDipakai[i] = false
}

rondeSelesai = false
keluarGame = false
skorRonde = 0

for !rondeSelesai {
    fmt.Print("Your tiles: ")
    for i = 0; i < 7; i++ {
        if !kartuDipakai[i] {
            fmt.Printf("(%d,%d) ",
kartuTanganPemain[i].KiriKartu,
kartuTanganPemain[i].KananKartu)
        } else {
            fmt.Print(" ")
        }
    }
    fmt.Println()

    fmt.Printf("Chain is (%d,%d) \t\t Now chain
is ", rantaiKartu[0].KiriKartu,
rantaiKartu[len(rantaiKartu)-1].KananKartu)

    for _, c := range rantaiKartu {
        fmt.Printf("(%d,%d)", c.KiriKartu,
c.KananKartu)
    }
}

```

```
        fmt.Println()

        fmt.Print("Decision: ")
        fmt.Scan(&keputusan)

        if keputusan == 9 {
            keluarGame = true
            rondeSelesai = true
            break
        } else if keputusan == 0 {
            fmt.Println("No more legal move")
            rondeSelesai = true
        } else {
            indeksKartu = keputusan
            if indeksKartu < 0 {
                indeksKartu = -indeksKartu
            }
            indeksKartu -= 1

            if indeksKartu < 0 || indeksKartu >= 7
|| kartuDipakai[indeksKartu] {
                fmt.Println("Invalid card choice!
Try again.")
                continue
            }

            kartuPilihan =
kartuTanganPemain[indeksKartu]

            if keputusan > 0 {
                ujungKanan =
rantaiKartu[len(rantaiKartu)-1].KananKartu
```

```

                                if kartuPilihan.KiriKartu ==
ujungKanan {
                                rantaiKartu =
append(rantaiKartu, kartuPilihan)
                                kartuDipakai[indeksKartu] =
true
                                skorRonde++
                                } else if kartuPilihan.KananKartu
== ujungKanan {
                                kartuPilihan.KiriKartu,
kartuPilihan.KananKartu = kartuPilihan.KananKartu,
kartuPilihan.KiriKartu
                                rantaiKartu =
append(rantaiKartu, kartuPilihan)
                                kartuDipakai[indeksKartu] =
true
                                skorRonde++
                                } else {
                                fmt.Println("Card doesn't match
the right end!")
                                }
                                } else {
                                ujungKiri =
rantaiKartu[0].KiriKartu
                                if kartuPilihan.KananKartu ==
ujungKiri {
                                rantaiKartu =
append([]Domino{kartuPilihan}, rantaiKartu...)
                                kartuDipakai[indeksKartu] =
true
                                skorRonde++
                                } else if kartuPilihan.KiriKartu ==
ujungKiri {

```

```

                                kartuPilihan.KiriKartu,
kartuPilihan.KananKartu = kartuPilihan.KananKartu,
kartuPilihan.KiriKartu

                                rantaiKartu =
append([],Domino{kartuPilihan}, rantaiKartu...)

                                kartuDipakai[indeksKartu] =
true

                                skorRonde++

                                } else {

                                fmt.Println("Card doesn't match
the left end!")

                                }

                                }

                                }

                                if skorRonde == 7 {
                                    rondeSelesai = true
                                }
                            }

                            if keluarGame {
                                break
                            }

                            skorTotal += skorRonde
                            totalRonde++

                            fmt.Printf("You got %d points\n\n", skorRonde)
                        }

                        fmt.Printf("Your last score is %d/%d\n", skorTotal,
totalRonde)

                        fmt.Println("Thank you for playing with us.")

                        if totalRonde > 0 {

```

```

        tingkatKemenangan = (float64(skorTotal) /
float64(totalRonde*7)) * 100

    } else {

        tingkatKemenangan = 0.0

    }

    fmt.Printf("Your winning rate is %.2f%%\t\t which
is total scores/(#rounds*7)\n", tingkatKemenangan)

}

```

The screenshot shows a Go IDE with a file named `sepasangKartu`. The code implements a domino game. The `main` function initializes variables for scores, rounds, player index, cards, and game state. It then enters a loop where it prints the current score and round, deals cards to the player, and prints the winning rate. The output window shows the following text:

```

109082500156
IF-13-02
Andromeda Alika Ramadhani

```

Output:

```

PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL18> go run "e:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL18\Soal123\n0123.go"
Your score is 0/0                               At the beginning scores=0, round=0
Dealing...                                     Shuffle the tiles for a new game
Your tiles: (2,3) (0,1) (4,5) (4,4) (1,6) (6,6) (3,6)
Chain is (2,5)                                Now chain is (2,5)
Decision: +3
Your tiles: (2,3) (0,1) (4,4) (1,6) (6,6) (3,6)
Chain is (2,4)                                Now chain is (2,5)(5,4)
Decision: +4
Your tiles: (2,3) (0,1) (1,6) (6,6) (3,6)
Chain is (2,4)                                Now chain is (2,5)(5,4)(4,4)
Decision: +1
Card doesn't match the right end!
Your tiles: (2,3) (0,1) (1,6) (6,6) (3,6)
Chain is (2,4)                                Now chain is (2,5)(5,4)(4,4)
Decision: 0
No more legal move
You got 2 points

Your score is 2/1                               At the beginning scores=2, round=1
Dealing...                                     Shuffle the tiles for a new game
Your tiles: (4,6) (5,5) (2,4) (2,5) (3,4) (2,2) (0,0)
Chain is (3,5)                                Now chain is (3,5)
Decision: 

```

Deskripsi Program:

Pada bagian akhir program, fungsi main() berperan sebagai pusat kendali yang menjalankan seluruh simulasi permainan Solitaire Gappleh. Di awal fungsi, semua variabel yang diperlukan dideklarasikan terlebih dahulu sesuai tipe datanya. Selanjutnya, program menginisialisasi proses pengacakan menggunakan `rand.Seed(time.Now().UnixNano())` agar susunan kartu yang dihasilkan selalu berbeda setiap kali permainan dijalankan.

Permainan utama dijalankan di dalam perulangan `for`. Pada setiap ronde, program membuat satu set kartu domino baru menggunakan fungsi `inisiasiDomino()`, kemudian mengacaknya dengan `kocokKartu()`. Setelah itu, pemain dibagikan 7 kartu yang disimpan dalam array `kartuTanganPemain` melalui fungsi `ambilKartu()`. Satu kartu tambahan diambil sebagai `kartuMulai` dan ditempatkan pada meja permainan yang disimpan dalam slice `rantaiKartu`. Pada saat yang sama, array `kartuDipakai` direset ke nilai `false` untuk menandakan bahwa semua kartu di tangan masih bisa dimainkan.

Saat ronde berlangsung, program menggunakan perulangan `for !rondeSelesai` untuk menangani setiap langkah pemain. Pada setiap giliran, program menampilkan kartu yang masih dimiliki pemain dan kondisi rantai kartu yang ada di meja. Pemain kemudian diminta memasukkan pilihan. Jika pemain memasukkan angka 9, permainan akan langsung dihentikan. Jika memasukkan angka 0, ronde dianggap selesai karena tidak ada langkah yang ingin dimainkan lagi. Selain itu, pemain dapat memasukkan nomor kartu yang ingin dipasang. Nilai positif digunakan untuk memasang kartu di ujung kanan rantai, sedangkan nilai negatif digunakan untuk memasang kartu di ujung kiri.

Sebelum kartu dipasang, program melakukan validasi untuk memastikan nomor kartu yang dipilih masih berada dalam rentang yang benar dan belum pernah digunakan sebelumnya. Setelah lolos validasi, program memeriksa apakah nilai pada kartu pilihan cocok dengan angka di ujung rantai yang dipilih. Jika kartu dapat dipasang, program akan menyesuaikan posisi kartu terlebih dahulu apabila diperlukan, misalnya dengan menukar sisi kiri dan kanan agar susunan domino tetap sesuai. Kartu kemudian ditambahkan ke rantai menggunakan fungsi `append()`.

Setiap kartu yang berhasil dipasang akan ditandai sebagai sudah digunakan pada array `kartuDipakai`, dan `skorRonde` akan bertambah satu. `Ronde` berakhir ketika pemain memilih berhenti, keluar dari permainan, atau semua kartu di tangan berhasil dipasang. Setelah `ronde` selesai, skor `ronde` ditambahkan ke `skorTotal` dan jumlah `ronde` yang telah dimainkan dicatat pada `totalRonde`.

Ketika pemain keluar dari permainan, program menghitung `tingkatKemenangan` berdasarkan total poin yang diperoleh dibandingkan dengan total poin maksimal yang dapat dicapai. Perhitungan dilakukan menggunakan tipe data `float64` agar hasil persentase lebih akurat. Terakhir, program menampilkan statistik dan persentase performa pemain di layar.

4. Soal4

Source Code:

```
package main

import (
    "bufio"
    "fmt"
    "os"
)

// 4.a) Operasi Dasar Mesin Karakter
var pitaKarakter string
var indeksPita int
var currentChar byte
var EOP bool

// Prosedur start
func start(input string) {
    pitaKarakter = input
    indeksPita = 0
    EOP = false

    if len(pitaKarakter) > 0 {
        currentChar = pitaKarakter[indeksPita]
```

```

        if currentChar == '.' {
            EOP = true
        }
    } else {
        EOP = true
    }
}

// Prosedur maju
func maju() {
    if !EOP {
        indeksPita++
        if indeksPita < len(pitaKarakter) {
            currentChar = pitaKarakter[indeksPita]
            if currentChar == '.' {
                EOP = true
            }
        } else {
            EOP = true
        }
    }
}

// Fungsi eop
func eop() bool {
    return EOP
}

// Fungsi cc
func cc() byte {
    return currentChar
}

```

```

}

// 4.b) Algoritma
// Point a & b = Membaca seluruh karakter dan
menghitung berapa banyak karakter terbaca
func hitungSemuaKarakter() int {
    var hitung int = 0

    for !eop() {
        hitung++
        maju()
    }
    return hitung
}

// Point c = Menghitung banyak huruf "A" yang terbaca
func hitungHurufA() int {
    var hitungA int = 0

    for !eop() {
        if cc() == 'A' || cc() == 'a' {
            hitungA++
        }
        maju()
    }
    return hitungA
}

// Point d = Menghitung frekuensi kemunculan huruf "A"
func hitungFrekuensiA() float64 {
    var hitungA int = 0

```

```

var hitungTotal int = 0

for !eop() {
    hitungTotal++
    if cc() == 'A' || cc() == 'a' {
        hitungA++
    }
    maju()
}

if hitungTotal == 0 {
    return 0.0
}

return float64(hitungA) / float64(hitungTotal)
}

// Point e = Menghitung ada berapa kata "LE" (pasangan
berurutan 'L' dan 'E')
func hitungKataLE() int {
    var hitungLE int = 0
    var lastChar byte

    if !eop() {
        lastChar = cc()
        maju()
    }

    for !eop() {
        if (lastChar == 'L' || lastChar == 'l') &&
(cc() == 'E' || cc() == 'e') {
            hitungLE++
        }
    }
}

```

```

        lastChar = cc()

        maju()

    }

    return hitungLE
}

func main() {

    var pembaca *bufio.Scanner =
bufio.NewScanner(os.Stdin)

    var teksKalimat string
    var totalA, totalLE int

    fmt.Print("Masukkan teks: ")

    if pembaca.Scan() {

        teksKalimat = pembaca.Text()

        fmt.Println("Hasil analisis: ")

        start(teksKalimat)

        totalA = hitungHurufA()

        fmt.Printf("Jumlah huruf 'A' yang terbaca:
%d\n", totalA)

        start(teksKalimat)

        totalLE = hitungKataLE()

        fmt.Printf("Jumlah kata 'LE' yang terbaca:
%d\n", totalLE)

    }

}

```

```
Soal4 > go mesinAbstrakKarakter.go > main
1 package main
2
3 import (
4     "bufio"
5     "fmt"
6     "os"
7 )
8
9 // 4.a) Operasi Dasar Mesin Karakter
10 var pitaKarakter string
11 var indeksPita int
12 var currentChar byte
13 var EOP bool
14
15 // Prosedur start
16 func start(input string) {
17     pitaKarakter = input
18     indeksPita = 0
19     EOP = false
20
21     if len(pitaKarakter) > 0 {
22         currentChar = pitaKarakter[indeksPita]
23         if currentChar == '.' {
24             EOP = true
25         }
26     } else {
27         EOP = true
28     }
29 }
```

109082500156
IF-13-02
Andromeda Alika Ramadhani

Ln 3, Col 26 47 characters AA Forma... 100% Windows (CI) UTF-8

connected to Discord Ln 136, Col 67 Tab Size: 4 UTF-8 CRLF Go 1.25.1 Go Live

Output:

```
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL18> go run "e:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL18\Soal14\mesinAbstrakKarakter.go"
Masukkan teks: LEMONADE DAN LALAPAN JUGA LELETASTIS
Hasil analisis:
Jumlah huruf 'A' yang terbaca: 7
Jumlah kata 'LE' yang terbaca: 3
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL18> go run "e:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL18\Soal14\mesinAbstrakKarakter.go"
Masukkan teks: Aku makan pecel lele dengan lalapan dan minum lemonade
Hasil analisis:
Jumlah huruf 'A' yang terbaca: 9
Jumlah kata 'LE' yang terbaca: 3
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\LATSOL MODUL18>
```

Deskripsi Program:

Pada bagian program ini, dibuat sebuah mesin pita karakter yang digunakan untuk membaca dan memproses teks secara berurutan, satu karakter demi satu karakter. Mesin ini memiliki empat variabel global, yaitu pitaKarakter untuk menyimpan teks input, indeksPita sebagai penunjuk posisi karakter yang sedang dibaca, currentChar untuk menyimpan karakter aktif saat ini, dan EOP (End of Tape) sebagai penanda bahwa proses pembacaan telah mencapai akhir teks.

Untuk menjalankan mesin pita, dibuat beberapa subprogram dasar. Prosedur start() digunakan untuk memulai pembacaan dengan memasukkan teks ke pita, mengatur indeks ke posisi awal, dan memeriksa apakah pita kosong atau langsung berakhir. Selanjutnya, prosedur maju() digunakan untuk menggeser pembaca ke karakter berikutnya dengan menambah nilai indeks dan memperbarui isi currentChar. Selain itu, terdapat fungsi eop() untuk mengecek apakah pembacaan sudah mencapai akhir pita dan fungsi cc() untuk mengambil karakter yang sedang dibaca saat itu.

Berdasarkan fungsi-fungsi dasar tersebut, dibuat beberapa fungsi untuk menganalisis isi teks. Fungsi `hitungSemuaKarakter()` digunakan untuk menghitung jumlah seluruh karakter dalam teks dengan membaca pita dari awal hingga akhir. Kemudian, fungsi `hitungHurufA()` digunakan untuk menghitung banyaknya huruf A atau a yang muncul pada teks. Setiap kali karakter yang dibaca sesuai dengan kondisi tersebut, nilai penghitung akan bertambah satu.

Selanjutnya, fungsi `hitungFrekuensiA()` digunakan untuk menghitung persentase kemunculan huruf A terhadap total karakter dalam teks. Perhitungan dilakukan menggunakan tipe data `float64` agar hasil pembagian lebih akurat dan tidak mengalami masalah pembagian bilangan bulat.

Analisis yang lebih lanjut dilakukan pada fungsi `hitungKataLE()`. Fungsi ini bertujuan menghitung berapa kali pasangan huruf "LE" muncul secara berurutan di dalam teks. Untuk melakukannya, digunakan variabel tambahan `lastChar` yang menyimpan karakter sebelumnya. Pada setiap langkah, program membandingkan `lastChar` dengan karakter yang sedang dibaca. Jika ditemukan kombinasi huruf L atau l yang diikuti oleh E atau e, maka penghitung `hitungLE` akan bertambah satu. Setelah itu, proses dilanjutkan hingga akhir pita.

Seluruh proses dijalankan melalui fungsi `main()`. Pada bagian ini, program menggunakan `bufio.NewScanner` untuk menerima input berupa kalimat lengkap dari pengguna, termasuk yang mengandung spasi. Setelah teks dimasukkan, program memanggil prosedur `start()` untuk memulai analisis. Mesin pita dijalankan terlebih dahulu untuk menghitung jumlah huruf A, kemudian diinisialisasi ulang menggunakan `start()` agar posisi pembacaan kembali ke awal sebelum menghitung jumlah pasangan huruf "LE".

Pemisahan antara mesin pita, fungsi analisis, dan fungsi utama membuat program lebih terstruktur dan mudah dipahami. Selain itu, pendekatan ini juga memudahkan pengembangan serta memastikan data yang sama dapat dianalisis berulang kali tanpa mengubah isi teks asli.

DAFTAR PUSTAKA

Febri Nova Lenti. (n.d.). *Mesin abstrak dalam pemrograman* [Dokumen PDF]. Scribd. <https://www.scribd.com/document/589538245/MESIN-ABSTRAK>

Program Studi S1 Informatika. (2026). *Modul algoritma dan pemrograman 2 IF REG 2026*. Universitas Telkom.